SISTEMAS OPERACIONAIS · PROGRAMAÇÃO PARALELA (P1)

Paralelização do clistats

Sequencial, processos e threads em C++17

Kaique Mazzocco

Aplicação e estratégia

Bárbara Hansen

Processos e comparação

Vinicius Coelho

Threads e validação

1 · APLICAÇÃO

clistats: estatísticas em linha de comando

Lê números separados por vírgula e calcula, por coluna, contagem, mínimo, média, máximo e desvio padrão, de forma rolante.

87 MB

arquivo CSV de entrada

2,53 mi

linhas (ID, idade, anos, salario, Despesas)

6,95 s

tempo da versão sequencial na medição

2 · ESTRATÉGIA

Divisão por dados em três etapas

1

Fatiar

O arquivo é cortado em fatias de bytes parecidas; cada corte avança até o próximo \n.

2

Calcular

Cada trabalhador gera estatísticas parciais da sua fatia, sem depender dos outros.

3

Combinar

Contagens somadas, mínimo e máximo globais, média e variância ponderadas.

A mesma divisão vale para processos e threads. Muda o mecanismo de execução.

3 · PROCESSOS

fork() e memória compartilhada

Mapeamento

O pai mapeia o arquivo com mmap (somente leitura) e cria a região MAP_SHARED | MAP_ANONYMOUS, com um slot por filho.

Execução

Com 4 processos, os filhos 1, 2 e 3 pegam as fatias 0, 1 e 2. O pai processa a fatia 3 enquanto espera.

Consolidação

Cada filho grava o parcial no seu slot. O pai espera com waitpid() e combina os resultados.

Sem mutex: cada filho escreve só no seu slot. Barreiras release/acquire garantem a visibilidade.

4 · THREADS

std::thread e seção crítica curta

Criação

A thread principal cria N std::thread, cada uma com uma fatia do arquivo.

Acúmulo local

O parsing roda em variáveis locais, sem travar nada.

Mesclagem

Ao terminar, a thread entra na seção crítica e mescla o parcial sob std::mutex (lock_guard).

A principal espera com join() e imprime o resultado. Mutex curto, pouca contenção.

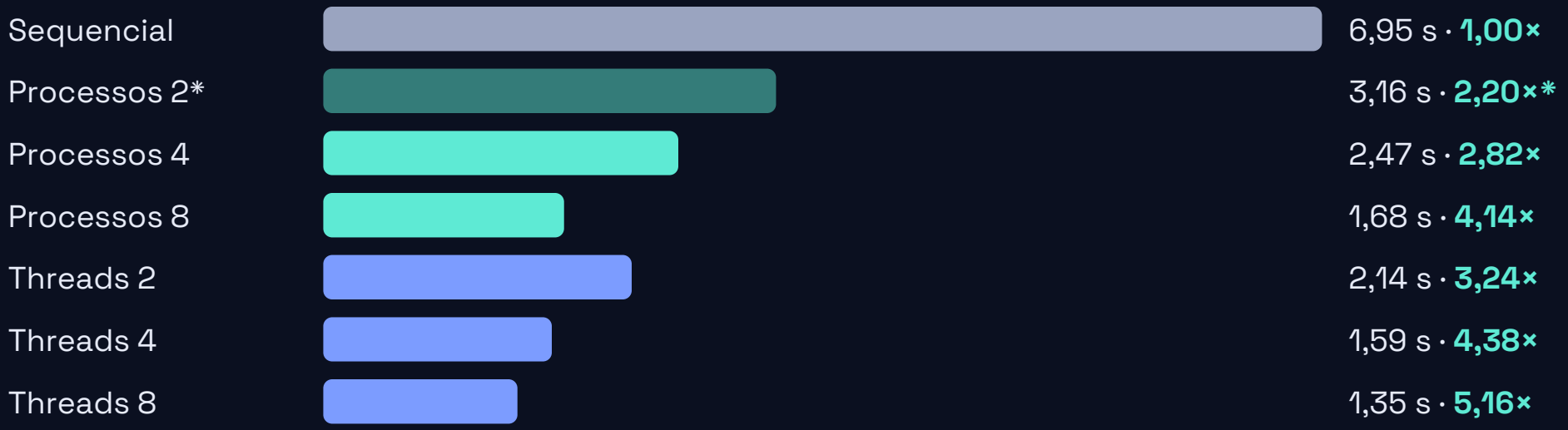
5 · COMPARAÇÃO

Processos x threads

Critério	Processos	Threads
Criação	fork() (mais cara)	std::thread (mais barata)
Memória	mmap compartilhada explícita	heap compartilhada
Sincronização	slots próprios + waitpid	std::mutex + join
Risco	dados só via memória compartilhada	condição de corrida sem mutex

6 · RESULTADOS

Tempo e speedup por versão



* Com 2 processos a contagem diverge do sequencial (2.459.165 contra 2.530.330), então o speedup não é comparável. Medição única por configuração, fora do container. O sequencial usa um parser mais lento, e o speedup pode estar superestimado.

Threads equivalentes; processos em correção

Equivalência

Threads (2, 4, 8): saída idêntica ao sequencial. Processos: 4 e 8 corretos no modo menu; com 2 e no modo -i ainda divergem.

Container

Dockerfile com Ubuntu 24.04 e g++. Testes com --cpus=1, 2 e 4.

Resultados

Com 8 trabalhadores: threads 1,35 s (5,16×) e processos 1,68 s (4,14×), contra 6,95 s do sequencial. Medições no container ainda por fazer.

Futuro: nº de trabalhadores como parâmetro e mesmo parser em todas as versões.

8 · CONCLUSÕES

O que aprendemos com o clistats

Divisão por dados

Cada fatia é independente e o custo de combinar os parciais é pequeno.

Processos e threads

A divisão de trabalho é a mesma. Mudam o custo de criação e a forma de compartilhar dados.

Resultados medidos

Threads foram mais rápidas em todas as quantidades: 5,16× com 8 threads, contra 4,14× dos processos. O efeito do limite de CPU no container ainda precisa ser medido.

FIM

Obrigado!

Perguntas?

Referências

clistats, dpmcmlxxvi

Projeto open source usado como base.